

An Agent-Oriented Framework to Support Rapid Early-Stage Object-Oriented Design

Yilong Yang^{1*} and Zhen Tian¹

^{1*}School of Software, Beihang University, Beijing, 100191, China.

*Corresponding author(s). E-mail(s): yilongyang@buaa.edu.cn;
Contributing authors: zhentian1@buaa.edu.cn;

Abstract

Object-Oriented (OO) design, as a cornerstone methodology in software engineering, has been playing a pivotal role in software development, during which OO design models are often crafted. However, the challenge is always on intensive cognitive effort and expertise required for modeling. In the past, automatically generating models from requirements has been considered a promising direction to overcome this challenge. The model-driven engineering (MDA) community has intensively made effort towards this direction. Though some progresses have been made with the help of natural language processing (NLP) techniques, fully automated solutions are rare due to the limitation of NLP techniques themselves, and generated models are not full correct and/or complete. Towards this direction, in this paper, we introduce an agent-oriented methodology with tool support, named RapidOOD, which leverages large language models (LLMs) for the generation, quality assessment, and refactoring of OO design models, by taking requirements models as the input. Our evaluation of RapidOOD with six requirements models demonstrates that automatically generated and subsequently optimized design models achieve 89.20% (without human) and 94.56% (human-in-the-loop collaboration) of expert-level quality metrics, closely match models crafted manually by experts. Furthermore, our approach achieves 1.39 times the efficiency of the manual method.

Keywords: Object-Oriented, Agent-Oriented, Model Generation, Model Refactoring, Large Language Model

1 Introduction

Software design is a fundamental part of software development, covering two primary stages: architectural design and detailed design[1]. The object-oriented design method, which facilitates the creation of different abstract types during the design process, is more intuitive than the process-oriented design method and has garnered considerable attention. After decades of evolution, object-oriented design has become one of the main approaches to software design and development[2]. Utilizing models is a vital supplementary technique in software design. Since textual descriptions are challenging to employ for development, review, and maintenance, abstraction through graphical modeling languages is used to enhance communication between customers and developers, support requirements validation, facilitate architectural and detailed design, aid in prototyping, enable code generation, and more. With Unified Modeling Language(UML), requirements can be modeled as a lightweight formal requirements model that consists of a use case diagram, a conceptual class diagram, use case definitions specified by system sequence diagrams, and the contracts of their system operations[3–5]. A system operation contract is formally specified by a pair of pre- and post-conditions in Object Constraint Language (OCL). Designs can be modeled using class diagrams and sequence diagrams. Works in requirements phase like RM2PT[6–9] provides features for requirements modeling, analysis, and automated prototype generation from requirements models. Users can rapidly validate and evolve functional requirements through RM2PT to produce high-quality validated requirements models. The validated requirements model can be used as input to the design phase.

Object-oriented design in software development is a knowledge and effort-intensive activity. Designers must invest significant time and effort to create an appropriate design solution for given requirements. Different scenarios and requirements necessitate different design solutions and trade-offs, often depending on the designer’s experience and subjective judgment. Additionally, evaluating the design quality early in the project is challenging and usually not feasible.

This paper aims to reduce the time and effort required during the software design phase through automated methods, lessen the dependence on the designer’s expertise, and provide a quantifiable and comprehensible way to reflect design quality early in the project. The main difficulties and challenges in generating design from requirements and subsequently evaluating and optimizing the initial design include:

1. Requirements describe the system’s external behavior, while design describes system’s internal implementation. Design is a concretization and refinement of requirements, containing more detailed information. It is challenging to supplement the missing information needed to transition from requirements to design.
2. Contracts describe the system state constraints before and after executing operations. Key challenges include determining whether there is a mapping from system states to system implementations (combinations of OO basic operations) and how to map these into operation sequences automatically.
3. In practice, design often requires multiple iterations of quality assessment and refactoring. A critical challenge is adopting suitable iteration strategies and methods to ensure iterations progress toward improved design.

These challenges highlight the complexity of software design, underscoring the potential benefits of automating certain aspects of the design process to improve efficiency, quality, and adaptability to changing technological trends.

This paper presents an agent-oriented methodology named RapidOOD. It supports the rapid generation, quality assessment, and refactoring of object-oriented designs. This methodology integrates model-driven rule-based agents and LLMs Agents. RapidOOD takes lightweight formal requirement models[10] as input, automatically generates initial object-oriented design models, and computes design quality metrics based on the model elements. It can offer simple rule-based refactoring suggestions based on these quality metrics, and it is also capable of providing complex refactoring recommendations based on LLMs. Designers can iterate on the design model regarding the quality metrics and refactoring suggestions, where each modification’s impact on design quality is immediately reflected through the metrics. A relatively high-quality design can be quickly achieved by automating generation, quality assessment, and refactoring suggestions.

We have developed a tool to automate the method. This method was evaluated using six existing requirement models. In terms of generating design models, the main focus was on assessing the coverage of the proposed rules, with the quality assessment results represented by the ratio of successfully generated class diagram elements and sequence diagram elements. The quality of the automatically generated design models and the quality of the design models after a few iterations were evaluated against the quality of models manually constructed by experts. The time savings achieved by adopting this method were also assessed. Overall, the experimental results were satisfactory.

The contributions of the paper are as follows:

1. **An Agent-Oriented Approach that Integrates Model-Driven Rule-Based Agents and LLM Agents:** This paper presents an agent-oriented methodology which facilitates the rapid generation, assessment, and refactoring of object-oriented designs. This methodology combines model-driven rule-based agents and LLM agents. The software design process is partitioned into an initial design phase featuring low degrees of freedom, a quality assessment phase, and a refactoring phase with high degrees of freedom. During the phases with low degrees of freedom, rule-based Agents are utilized to avoid problems like hallucinations resulting from the premature application of LLMs. In the phases with high degrees of freedom, LLMs are incorporated to fully exploit their knowledge-emergence capabilities.
2. **A Method for Analyzing the Executability of OCL Contracts and Path Reasoning:** Our work introduces an automated approach for analyzing the executability of a given OCL contract and determine a reasonable implementation path. This method effectively bridges the gap between requirements and design, providing an efficient, automated pathway for object-oriented software design.
3. **Design refactoring with Embedding quality assessment Results in LLM Prompts:** Utilizing a model-driven methodology, this research advances design quality quality assessment from code-based assessments to an innovative model-based framework. The design quality quality assessment results obtained through

the model-driven method are embedded into LLM Prompts to guide the LLM to propose corresponding refactoring suggestions

The rest of this article is organized as follows. Section 2 discusses the related work. Section 3 presents the background of our approach. Section 4 introduces RapidOOD in detail, including how to automatically generate design models from requirements models, how to evaluate design quality, and how to give refactoring suggestions. Section 5 presents the evaluation of our approach. Section 6 concludes this article.

2 Related Work

According to the existing research, the automatic generation of software design models can be divided into two types according to the input and generation methods. In some research, the input is mainly the requirement described by natural language. The requirement description is processed using the related natural language processing methods to generate the design model. In other research, the input is the requirement model, which uses the model-driven method to convert requirement models to design models by defining the transformation rules. Each study presents requirements and design models differently but contains consistent information. Requirement models generally describe what entities exist in the system, the relationships between them, what services the system can provide to users, what operations the system can complete, and the system's state before and after operations, which information users can obtain from outside the system. Design models generally describe information related to system implementation, such as how classes within the system are organized, how classes relate to each other, and how system operations are accomplished by instances of classes in collaboration.

2.1 Design Generation from Requirements Description

In terms of generating design models from requirements described in natural language, Mala et al.[11] proposed a method to extract object-oriented elements from natural language. This method divides the original requirements into sentences, labels each word in the sentence, and then recombines nouns and verbs through a combiner based on simple phrase grammar to obtain a relatively structured requirements directory. Then, words with different parts of speech are translated into classes, attributes, and use cases through the defined conversion rules. For example, nouns can be translated into classes and attributes of classes. The disadvantage of this work is that the identified classes may be more than the actual classes. The generated class does not contain methods, cannot describe the interaction between objects within the system, and does not contain the information corresponding to the sequence diagram.

To reduce the ambiguity caused by the rich semantics of the natural language, V. s. sharma[12] and others proposed a method to extract functional designs from relatively structured restricted natural languages and generate UML class diagrams and implemented this method as a tool FDCT (functional design creation tool), which was integrated into IBM Rational Software Architect as a plug-in. The original requirement text is processed by the tool rat produced by its previous work to obtain specific

types of strings. The heuristic rules proposed by it are applied to these strings, and finally, a class diagram containing classes, methods and relationships between classes is obtained. Its original requirement text is limited by user-defined vocabulary, and the definition of vocabulary is complex, which is not easy compared with requirement modeling. Only some types of requirements are supported. The generated class diagram does not contain attributes. The internal behavior of the system was not explicitly designed.

Approach *aToucan*[13] automatically generates analysis models consisting of class, sequence, and activity diagrams from requirement models containing natural language. Although *aToucan* can generate sequence diagrams and activity diagrams, their content is limited to human-system interactions and lacks designs to collaborate objects within the system.

2.2 Design Generation from Requirements Model

The most related research to this study is related work on automatically generating design models from lightweight formal requirements models. Steve et al. [14] introduced the lightweight application of formal methods in requirements modeling of spacecraft fault protection systems through three case studies and explained the benefits of introducing lightweight formalization in the requirements stage. Xiaoshan Li et al. [15] proposed a formal method for requirements modeling using UML. This method was further developed by ZhiMing Liu et al.[16–18], who proposed the Refinement of Component and Object Systems (rCOS) method. This paper provides a formal model definition for the UML-based Rational Unified Process (RUP) development process, which can facilitate the introduction of model-driven methods in the requirements, design, and verification phases of component-based object-oriented systems. And better guarantee consistency and correctness, and also provide a basis for introducing automated methods. The manual process from requirements model to a detailed functional design model is introduced in his work. In terms of design model generation from the formal requirements model, Nora Koch[19] uses the model transformation method to transform the requirement model into the design model for the web system. The meta-model of the input requirement model is the Webre meta-model derived from summarizing the concepts that need to be used in web system modeling. This meta-model refines the UML meta-model. UWE meta-model defines the output design model. There is a simple mapping relationship between elements in the UWE meta-model and classes and the relationship between classes. Web system design is described in presentation, navigation, and content. The conversion rules of the two models are defined in QVT language, which can realize bidirectional conversion and comply with OMG standards. This research ultimately realizes the transformation from the web system requirement model to the design model and proves the feasibility of automatically generating the design model based on the model transformation. However, some things could be improved: this research is mainly aimed at the GUI model and does not involve the system operation encapsulation and the interaction of internal objects of the system. G. Mussbacher et al.[20] proposed a transformation method of an oriented model for a reactive system. From the scenario-based requirement model aoucm (aspect-oriented use case maps) to the UML-based design model

202 RAM (reusable aspect models), it further confirms the feasibility of transforming the
203 requirement model into the design model on different systems.

204 2.3 OO Design Quality Metrics and Quality Optimization

205 There are four groups of widely accepted design metrics (CK[21] LK[22], MOOD[23],
206 and QMOOD[24]) for software object-oriented design quality metrics and assessment.
207 The latest QMOOD is an improved hierarchical model for evaluating high-level design
208 quality attributes in object-oriented design. The model relates low-level statistical
209 metrics of design with design properties (such as encapsulation, modularity, coupling,
210 and cohesion) and high-quality properties (such as reusability, flexibility, and com-
211 plexity). Pham[25] et al. used the above four groups of design metrics to propose a
212 set of unified metrics covering many aspects of object-oriented software design. Out-
213 side of the set of quality metrics, there is ongoing exploration into different aspects
214 of object-oriented design. In the research of design quality assessment using design
215 metrics, the existing problem is that there are empirical values when the lower-level
216 quality metrics or features are integrated into the upper-level quality attributes, and
217 these empirical values cannot be automatically adjusted according to the system type
218 and domain knowledge. Meng Yan[26] et al. conducted a systematic mapping study
219 on the related research of quality assessment model, and an important conclusion was
220 that there was a lack of tool support in software design assessment.

221 Regarding software design optimization, there are two paths: quality-driven model
222 transformation and search algorithm. Insfran [27] et al. proposed design guidelines for
223 developing quality-driven model transformations. By using quality models to express
224 quality attributes and transformation models to express alternative model transfor-
225 mations and their effects on quality attributes, transformation designers are allowed
226 to use quality attributes as decision factors for choosing alternative transformations.
227 The correctness of this approach depends on its transformation model and is not flex-
228 ible enough. The use of search algorithms for software design optimization is mainly
229 the use of genetic algorithms [28–30], including single-objective and multi-objective
230 genetic algorithms. Design optimization schemes proposed using model transforma-
231 tion or search algorithms also fail to incorporate system type and domain knowledge
232 and are less effective.

233 2.4 Summary of Related Work

- 234 • Some existing works provide supplementary information about the system’s internal
235 static implementation (including how classes are organized and what operations
236 they have) but fail to include the system’s internal dynamic design information
237 (mainly how objects within the system interact to complete operations).
- 238 • Existing works do not consider the executability of system operation contracts and
239 the sequencing of atomic operations.
- 240 • Existing works do not explain the impact of each iteration on design quality in
241 an easily understandable way, nor do they integrate the quality assessment and
242 refactoring phases in an automated manner.

- To the best of our knowledge, existing work has not effectively combined model-driven methods with LLMs for application in software design tasks.

2.5 Comparison to RM2DM

Our previous tool, RM2PT, facilitated rapid prototyping from requirement models to swiftly confirm requirements. Building on these confirmed requirements, our previous work, RM2DM[31], introduces a tool that can swiftly generate design models adhering to the anemic model design concept from the confirmed requirements. Despite the anemic model design being considered an anti-pattern, it is extensively employed in practical project development, such as in Java EE and Spring, which are both based on the anemic model concept. In comparison to our previous work, the method and tool presented in this paper offer the following:

- *Detailed transformation rules from requirements to sequence diagrams in design models:* Our previous work only provided tool implementation without explicitly stating the rules for generating sequence diagrams in the design models. In this paper, we detail the transformation rules and their use cases.
- *Integration of automatic quality assessment and refactoring methods based on the generated design models:* Our previous work could only generate a simple initial design model. In this method, we use QMOOD metrics and some rules to provide automatic quality assessment and refactoring capabilities, further addressing the issue of design automation. Corresponding tool support is also provided based on Sirius.

3 Preliminary

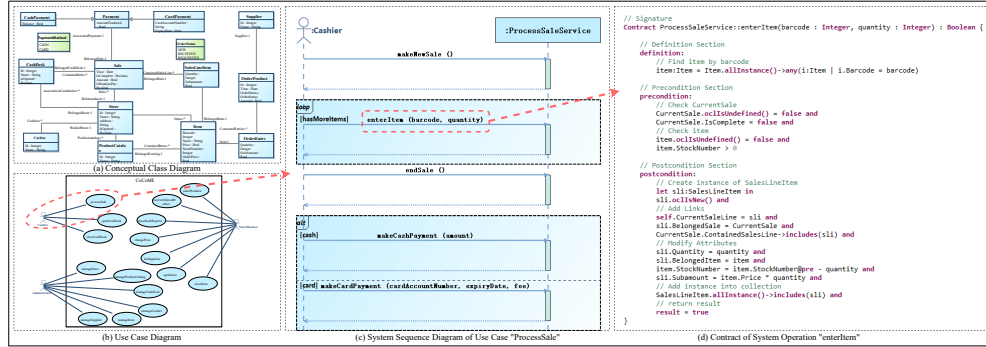


Fig. 1 Requirements Model

3.1 Formal Requirements Model

The requirement model used in our approach includes use case diagrams, conceptual class diagrams, system sequences diagrams, and contracts of system operations. Figure 1 shows an example of requirements models. Introducing lightweight formal methods during critical phases of software requirement modeling can enhance software reliability and reduce the cost of crucial software without significantly burdening the software process[10]. The formal semantics of the requirement models adopted in this study have been proposed and utilized in research [3–5, 8].

- **Use Case Diagram:** Use case diagrams illustrate interactions between users and the system, detailing the system’s services and user capabilities. Unlike system operations, which often collaborate to fulfill a use case, use cases aim to accomplish user goals. The concept of an Actor is crucial, as it represents users or external systems interacting with the primary system.
- **Conceptual Class Diagram:** Conceptual class diagrams depict relationships within the problem domain, where classes represent concepts, associations define class relationships and class attributes signify properties. Unlike design model class diagrams, these do not detail operations or system encapsulation strategies. In the requirements phase, the architecture of the system and the assignment of responsibilities to classes are not solved.
- **System Sequence Diagrams:** System sequence diagrams outline use case implementation processes, identifying required participants, their initiated system interactions, and the sequence of these events. They treat the system as a black box, focusing on input and output without detailing internal object interactions.
- **Contracts of System Operations:** System operation contracts detail individual operations, specifying preconditions and post-conditions for system states. They guide adjustments post-operation, including object and association management and property modifications. Written in Object Constraint Language (OCL), these contracts provide a formal yet accessible means to define system constraints.

3.2 OO Design Model

The design model comprises class diagrams and sequence diagrams. Class diagrams outline the system’s static characteristics, detailing classes, their interrelations, attributes, and operations. Sequence diagrams capture the dynamic interactions among objects, depicting the sequence of events. An example of a class diagram is shown in Figure 2 and 3, and a sequence diagram in Figure 4.

3.2.1 Class Diagram

Class diagrams are typically used during the system design phase to provide a comprehensive view of the system’s architecture, including class definitions, their relationships (such as aggregation, composition, and inheritance), and their attributes and operations. These diagrams not only illustrate the roles and responsibilities of each class within the system but also clarify encapsulation strategies and interface

design. As such, class diagrams are essential for understanding the system’s internal structure, information flow, and collaboration among classes.

In contrast, conceptual class diagrams are primarily employed for modeling the problem domain, often appearing in the requirements analysis phase. The focus of a conceptual class diagram is to abstractly represent domain concepts, where classes denote real-world entities rather than specific code implementations. These diagrams concentrate on class relationships and attributes, without addressing class operations or system implementation details. Therefore, conceptual class diagrams do not reveal encapsulation strategies or the allocation of responsibilities to objects. Their purpose is to assist designers in understanding the fundamental concepts within the problem domain and the relationships between them, without delving into implementation specifics.

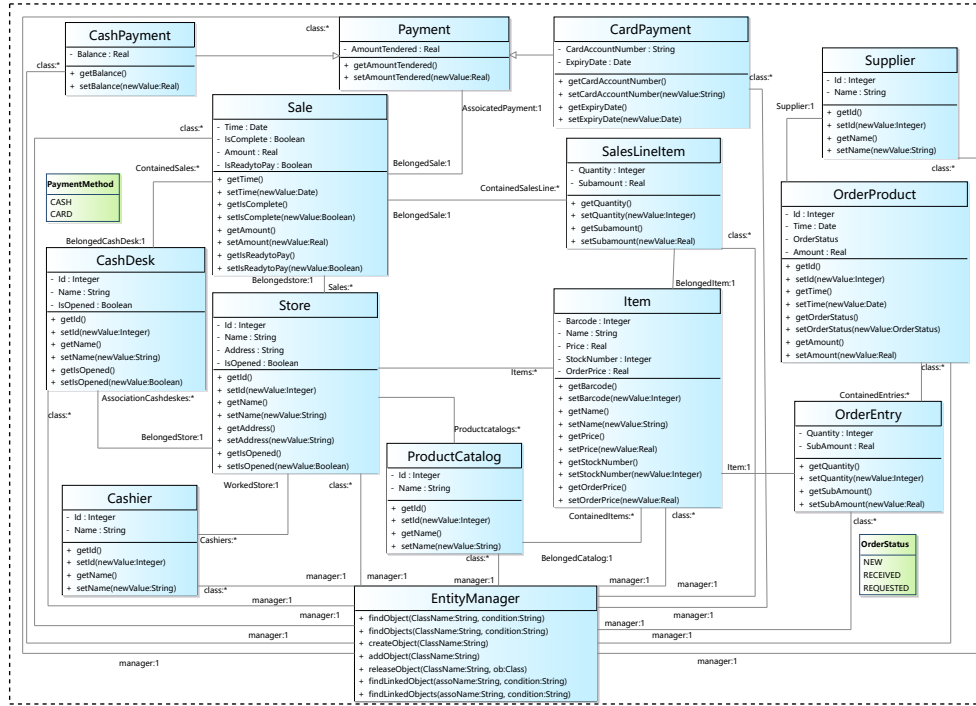
3.2.2 Sequence Diagram

Sequence diagrams describe the interactions between objects over time, illustrating how messages are exchanged to accomplish system functionality. These diagrams provide a detailed depiction of the operations performed by each object at different points in time and emphasize the collaborative relationships between objects. Sequence diagrams are commonly used during the design phase to refine the behaviors of internal system objects, the sequence of messages, and interaction details.

By contrast, system sequence diagrams present a simplified view of use cases, focusing on the interactions between the system and external actors. In system sequence diagrams, the system is treated as a black box, with attention placed on the inputs received and the outputs generated, without describing the internal interactions between objects. System sequence diagrams emphasize the design of system interfaces and assist developers in understanding how the system should respond to external requests. Compared to sequence diagrams, system sequence diagrams operate at a higher level of abstraction, focusing more on a macro-level description of system behavior.

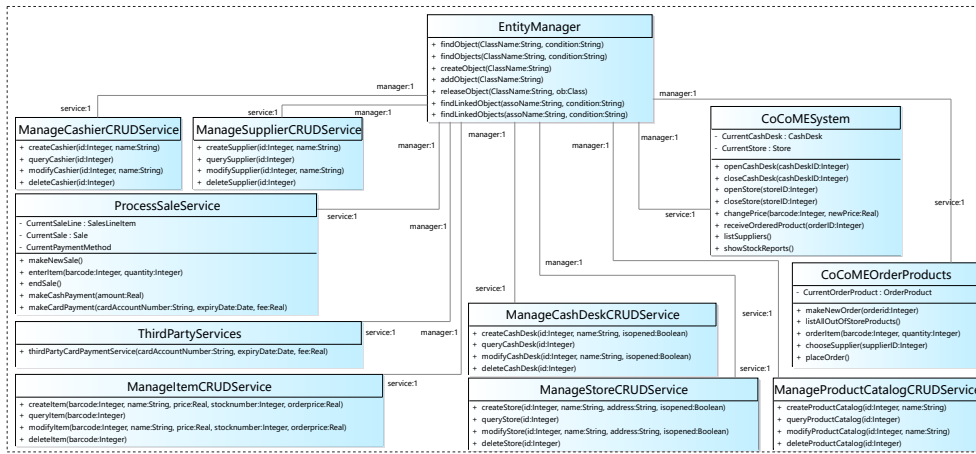
3.3 Anemic Domain Model

The anemic domain model, a term from Domain-Driven Design[32], delineates a simplified structure where the core domain model spans Domain, Service, and Repositories. Entities and Value Objects reside within the Domain layer, while domain services and logic for persisting and querying these objects are positioned in the Service and Repositories layers. Unlike a rich domain model where entities encapsulate data, operations, and business logic, the anemic model features entities with only data structures and basic operations—relegating business logic to the Service layer. Despite being considered a design anti-pattern, the anemic model is prevalent in practical coding, particularly in web projects. When the system business is relatively simple, mainly data addition, deletion, modification, and search, it is more appropriate to use



(a) Entity Classes of Generated Class Diagram

Fig. 2 Generated Class Diagram Entity Part



(b) Service Classes of Generated Class Diagram

Fig. 3 Generated Class Diagram Service Part

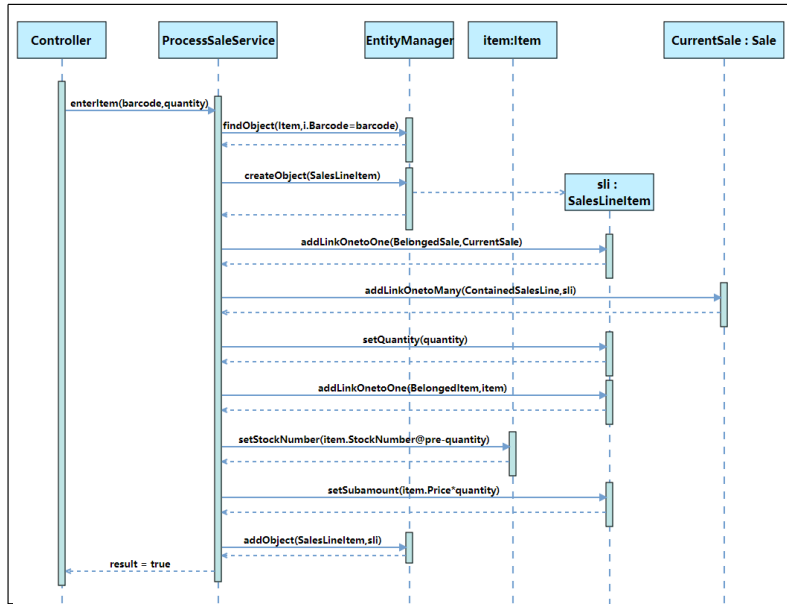


Fig. 4 Generated Sequence Diagram for System Operation “enterItem”

the anemic model than the domain model. Its widespread use is supported by significant frameworks such as Java EE¹ and Spring², underscoring its relevance in modern software development.

3.4 Rule-based Agents and LLMs-based Agents

Software agents and multi-agent systems have received considerable research attention since the 1980s [33]. An agent, as an autonomous system, is capable of interacting with other agents to fulfill its designed objectives. Multi-Agent Systems (MAS) consist of multiple autonomous entities, or agents, that interact within a shared environment to achieve individual or collective goals. In the research of agent-oriented software engineering, the primary focus is on how to utilize agents as fundamental building modules for designing and developing complex systems, with an emphasis on modularity, reusability, and software architecture. The agents therein typically operate based on predefined rules or logical reasoning, and their interactions mainly base on rule and deterministic protocols or algorithms. We refer to this kind of agent as Rule-based Agents.

Large Language Models are a class of deep learning models designed to understand and generate human-like text. LLMs leverage massive datasets and billions of parameters to achieve remarkable language comprehension and generation capabilities. Due to the rapid development of LLMs, LLMs-based agents have attracted significant

¹<https://www.oracle.com/java/technologies/java-ee-glance.html>

²<https://spring.io/>

attention. LLMs-based agents take LLMs as the core. Through reinforcement learning and function-calling techniques, such agents can automate complex tasks via the “perceive-decide-act” process.

4 Approach

This section describes our method for transforming requirement models into design models (including a class diagram and sequence diagrams), assessing design quality during the design phase, and offering straightforward refactoring recommendations.

4.1 Overview

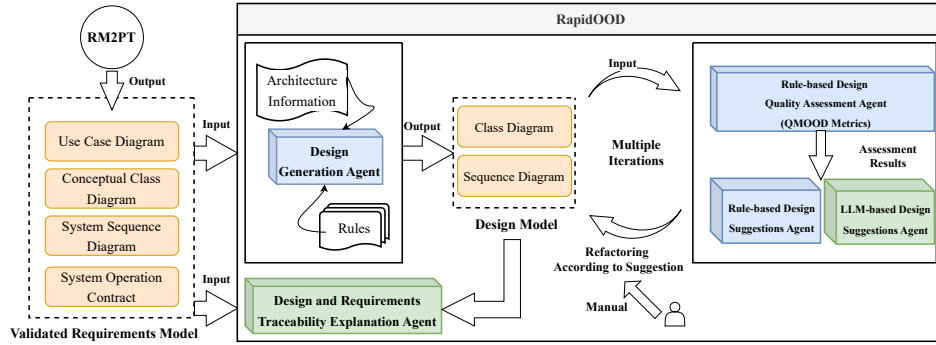


Fig. 5 Overview of RapidOOD

Our approach takes validated requirements models as input. Auto-prototyping work, such as RM2PT, assists in swiftly attaining a validated requirements model by automatically producing prototypes from requirement models, thereby facilitating incremental and agile requirements validation. We propose a set of rules and algorithms to generate the design models composed of a class diagram and a set of sequence diagrams. Based on the initial design models generated, we employ QMOOD, a hierarchical model for evaluating the quality of object-oriented designs, to assess the design quality. The results of the quality assessment metrics and the simple refactoring suggestions we propose are fed back to the users. Users can then manually iterate on the design based on this feedback, thereby swiftly achieving a relatively higher quality design.

In this paper, the agents are not solely centered around LLMs, instead, the framework incorporates a hybrid approach where agents are built upon both LLM-driven capabilities and traditional model-driven methods. While LLM-based agents excel in tasks requiring natural language understanding, reasoning, and adaptability, the model-driven agents leverage predefined rules, domain-specific models, and structured decision-making processes to handle tasks with deterministic logic or stringent performance requirements. This duality allows the system to balance the flexibility of LLMs

with the reliability and efficiency of traditional approaches, creating a more robust and versatile multi-agent framework.

In RapidOOD, we designed the Design Generation Agent, Design Quality Assessment Agent, and rule-based Design Suggestions Agent that provides simple recommendations using a model-driven, rule-based approach. The model-driven approach generates the initial design model from the requirements model, ensuring excellent traceability and the scale of the initial design. (As can be seen in the subsequent experimental section, the design models generated solely by LLMs are limited in scale and often lack essential relationships or operations.) For the quality assessment of the design, a quantitative method based on QMOOD Metrics was adopted. By statistically analyzing the model elements of the design model, high-level quality attribute metrics of the design model were obtained, such as reusability. For low quality attributes, rule-based design refactoring suggestions were provided. For example, parts with high coupling in the design model were highlighted and suggestions for refactoring were given.

We designed the LLM-Based Design Suggestions Agent and the Traceability Explanation Agent. The aim is to utilize the knowledge-emergence ability of LLMs to provide expert-level design refactoring suggestions and to offer easily understandable explanations for the traceability between the design and the requirements.

4.2 Design Model Generation Agent

4.2.1 Transformation Rules of Class Diagram Generation

Three transformation rules are proposed to automatically generate a class diagram from the requirement model .

Preserve the Classes and Relationships

The first rule is to preserve the relationships between classes and classes in the conceptual class diagram and add GET/SET methods corresponding to the class attributes. This design is based on the fact that the relationships between classes and classes in the conceptual class diagram in the requirement model are naturally consistent with the actual situation, and this part should remain the same; otherwise, relationship errors or missing relationships will occur.

Generate Service Class

The second rule is to encapsulate all use case-related system operations into a new Service class based on the use case diagram (including use cases) and the system sequence diagram (including system operations related to use cases). For a case, according to the information in the use case diagram and the system sequence diagram, a new class is extracted, which contains the class name, attributes, operations, and other information. It extracts class names from use case diagrams and attributes and operations from system sequence diagrams and system operation contracts.

The classes extracted by this rule will serve as the content of the Service layer and can be used to provide services to the outside world. Service will act as the initiator

429 of the system operation, and then the various classes within the system will work
 430 together to complete the various system operations.

431 ***Generate EntityManager Class***

432 The third rule involves the creation of a new EntityManager class, which serves as
 433 the central hub for all other classes. It is responsible for adding, deleting, modifying,
 434 and querying objects, and thus should have primitive operations related to these
 435 tasks. The operations owned by EntityManager are derived by summarizing the OCL
 436 statements in the system operation contract. 10 types of OCL expressions associated
 437 with EntityManager and 7 primitive operations are needed.

$$\begin{aligned} E_1 : obs : Set(ClassName) &= ClassName.allInstances() \\ E_2 : obs : Set(ClassName) &= ClassName.allInstances() \\ &- > select(o|conditions(o)) \end{aligned}$$

438 The meaning of statement 1 is to find instances of all ClassName classes in the
 439 system, and the meaning of statement 2 is to find instances of conditional conditions
 440 of all ClassName classes in the system. To complete the behavior described by the
 441 two OCL statements, the class EntityManager requires the operation findObjects().

$$\begin{aligned} E_3 : ob : ClassName &= ClassName.allInstances() \\ &- > any(o|conditions(o)) \end{aligned}$$

442 The meaning of statement 3 is to find an instance of a conditional condition of
 443 any ClassName class in the system. Therefore, the class EntityManager requires a
 444 findObject() operation.

$$\begin{aligned} E_4 : o : ClassName &= ob.assoName \\ E_5 : o : ClassName &= ob.assoName \\ &- > any(o|conditions(o)) \end{aligned}$$

445 Statement 4 means to find a single instance of a class in the system that has an
 446 assoName relationship with an ob class, and statement 5 means to find any instance
 447 of a conditional condition of a class that has an assoName relationship with an OB
 448 class in the system. Both statements find a single object based on the association
 449 relationship, so the EntityManager needs an operation findLinkedObject ().

$$\begin{aligned} E_6 : obs : Set(ClassName) &= ob.assoName \\ E_7 : obs : Set(ClassName) &= ob.assoName \\ &- > select(o|conditions(o)) \end{aligned}$$

450 The meaning of statement 6 is to find a collection of instances of a class in the
 451 system that has an assoName relationship with an ob class, and the meaning of
 452 statement 7 is to find a collection of instances of a conditional class in the system

453 that has an assoName relationship with an ob class, so the EnitityManager needs to
 454 be able to find a collection of objects based on the association relationship and needs
 455 to operate findLinkedObjects ().

$$E_8 : let ob : ClassName in ob.oclIsNew()$$

456 The meaning of statement 8 is to define an object OB for a ClassName, so the
 457 EnitityManager needs to be able to create instances of a particular class and needs
 458 to have an operation createObject ().

$$E_9 : ClassName.allInstances() - > includes(ob)$$

459 Statement 9 appears in the Postcondition section of the system operation contract,
 460 indicating that the OB object needs to be guaranteed to be included in the instance
 461 of ClassName after the system operation is executed, so EnitityManager needs to
 462 add the OB object to the list of instances of ClassName class, and the addObject ()
 463 operation is required.

$$E_{10} : ClassName.allInstances() - > excludes(ob)$$

464 Statement 10 appears in the Postcondition section of the system operation con-
 465 tract, indicating that you need to ensure that the ClassName instance after the system
 466 operation is executed does not contain an ob object, so EnitityManager needs to
 467 remove the OB object from the list of ClassName instances, and you need an operation
 468 releaseObject().

469 4.2.2 Transformation Rules of Sequence Diagram Generation

470 RapidOOD uses sequence diagrams to describe the collaboration of objects within the
 471 system. The main content of the sequence diagram is the messages passed between
 472 objects. Based on the anemic domain model, there is a controller in the system. The
 473 controller sends the invocation messages of system operations to the corresponding
 474 service class. The service class sends messages to the related objects inside the system,
 475 and the objects collaborate to complete the system operations and return. The primary
 476 challenge lies in the generation of messages between objects. Of the three components
 477 of the system operation contract, the OCL expressions in the precondition part define
 478 the system's required state before executing the operation. As these preconditions do
 479 not involve interactions between class instances, they are not converted into messages.
 480 The definition and post-condition of the system operation contract, on the other hand,
 481 describe the system's dynamic behavior. The OCL expressions in these sections can be
 482 transformed into messages in sequence diagrams. To facilitate this transformation, a
 483 set of 17 transformation rules has been established. Transformation rules are presented
 484 in this form:

$$Rule : \frac{OCL\ Expression}{Message\ Label,\ Sender,\ Receiver}$$

485 The transformation rule contains two parts: the above is an OCL expression in
 486 the contracts, and the bottom part is the generated message and the message's sender
 487 and receiver.

$$R1 : \frac{obs: Set (ClassName) = ClassName.allInstances()}{findObjects (ClassName), Service, EntityManager}$$

$$R2 : \frac{obs: Set (ClassName) = ClassName.allInstances() \rightarrow select (o \mid conditions(o))}{findObjects (ClassName, conditions (o)), Service, EntityManager}$$

$$R3 : \frac{ob: ClassName = ClassName.allInstances() \rightarrow any (o \mid conditions(o))}{findObject (ClassName, conditions (o)), Service, EntityManager}$$

$$R4 : \frac{o: ClassName = ob.assoName}{findLinkedObject (ob, assoName), Service, EntityManager}$$

$$R5 : \frac{o: ClassName = ob.assoName \rightarrow any (o \mid conditions(o))}{findLinkedObject (ob, assoName, conditions(o)), Service, EntityManager}$$

$$R6 : \frac{obs: Set (ClassName) = ob.assoName}{findLinkedObjects (ob, assoName), Service, EntityManager}$$

$$R7 : \frac{obs: Set (ClassName) = ob.assoName \rightarrow select (o \mid conditions(o))}{findLinkedObjects (ob, assoName, conditions(o)), Service, EntityManager}$$

Rules 1 to 7 are the conversion rules from the definition part of the OCL statement to the message. The meaning of this part of the statement has been introduced above. According to this part of the conversion rules, the sending end of the message is Service, and the receiving end is EntityManager.

$$R8 : \frac{let ob: ClassName in ob.oclIsNew()}{createObject (ClassName), EntityManager, ob}$$

The meaning of statement 8 is to generate an instance ob of the ClassName class. The operation of creating an object is initiated by the Service and completed by the EntityManager to generate an object ob. There are actually two request messages in this process. One is that the Service sends a message to the EntityManager to request the creation of an instance; the other is the creation message of the object ob created by the EntityManager.

$$R9 : \frac{ClassName.allInstances() \rightarrow includes (ob)}{addObject (ClassName, ob), Service, EntityManager}$$

$$R10 : \frac{ClassName.allInstances() \rightarrow excludes (ob)}{releaseObject (ClassName, ob), Service, EntityManager}$$

Statement 9 and statement 10 are the OCL statements of the postcondition part, and the related class is also EntityManager, which has been introduced in the previous article. For the messages obtained according to rules 9 and 10, the sending end is Service, and the receiving end is EntityManager.

$$R11 : \frac{ob.assoName \rightarrow includes (addOb)}{addLinkOnetoMany (assoName, addOb), Service, ob}$$

The meaning of statement 11 is to add the addOb object to the list of objects pointed to by the assoName relation of ob, corresponding to the atomic operation addLinkOnetoMany(), which is executed by the object ob.

$$R12 : \frac{ob.assoName \rightarrow excludes(removeOb)}{removeLinkOnetoMany(assoName, removeOb), Service, EntityManager}$$

505 The meaning of statement 12 is to remove the removeOb object from the list of
 506 objects pointed to by the assoName relation of ob, and the corresponding atomic
 507 operation is removeLinkOnetoOne(), which is executed by the object ob.

$$R13 : \frac{ob.assoName = addOb}{addLinkOnetoOne(assoName, addOb), Service, ob}$$

508 The meaning of statement 13 is to change the object pointed to by the assoName
 509 relation of ob to addOb, and the corresponding atomic operation is addLinkOne-
 510 toOne(), which is executed by the object ob.

$$R14 : \frac{ob.assoName = null}{removeLinkOnetoOne(assoName), Service, ob}$$

511 The meaning of statement 14 is to change the object pointed to by the assoName
 512 relation of ob to null, i.e., it does not point to any object, and the corresponding
 513 atomic operation is removeLinkOnetoOne(), which is executed by the object ob.

$$R15 : \frac{ob.attriName = mathExp}{setAttriName(mathExp), Service, ob}$$

514 The meaning of statement 15 is to change the value of the attriName attribute of
 515 ob to the value represented by mathExp, and the corresponding atomic operation is
 516 setAttribute(), which is executed by the object ob.

$$R16 : \frac{result = var}{return(var), Service, Controller}$$

517 The meaning of statement 16 is to return the result of the system operation.

$$R17 : \frac{ThirdPartyService.opName(vars)}{opName(vars), Service, ThirdPartyService}$$

518 The meaning of statement 17 is to call a method of a third-party service. For each
 519 OCL expression in the system operation contract, apply rules 1-7 if it belongs to the
 520 definition part of the contract and rules 8-17 if it belongs to the post-condition part.
 521 Determine the rules for the OCL expression based on its syntactic structure. Apply
 522 the rules to extract the label, the sending object and receiving object of the message
 523 from the OCL expression.

524 Algorithm 1 shows how to apply the rules to generate messages from system opera-
 525 tion contracts. For each OCL expression in the system operation contract, apply rules
 526 1-7 if it belongs to the definition part of the contract and rules 8-17 if it belongs to
 527 the post-condition part. Determine the rules for the OCL expression based on its syn-
 528 tactic structure. Apply the rules to extract the label, the sending object and receiving
 529 object of the message from the OCL expression.

530 Algorithm 2 describes how to order OCL statements to ensure that they are
 531 executable. It involves constructing a dependency graph and performing topological
 532 sorting to ensure that all dependencies are respected. The algorithm details are as
 533 follows.

534 **Initialize the Dependency Graph:** Create an empty directed graph $G = (V, E)$,
 535 where V represents the set of nodes (OCL statements) and E represents the set of
 536 directed edges (dependencies between statements).

Algorithm 1 Sequence Diagram Messages Generation

Input: System Operation Contract

Output: Sequence Diagram Messages

```
1: for each OCLExpression  $\in$  Contract do
2:   if OCLExpression  $\in$  definition then
3:     applyRule(1 – 7)
4:   else if OCLExpression  $\in$  postcondition then
5:     applyRule(8 – 17)
6:   end if
7: end for
8: procedure applyRule(ruleOrder)
9:   setMessageLabel()
10:  setMessageSender()
11:  setMessageReceiver()
```

537 **Add Nodes to the Graph:** For each OCL statement $s_i \in S$ (where S is the set
538 of OCL statements), add s_i as a node in V .

539 **Add Edges to the Graph:** For each OCL statement $s_i \in S$, identify its depen-
540 dencies. For each dependency s_j of s_i , add a directed edge (s_j, s_i) to E . This indicates
541 that s_i depends on s_j .

542 **Topological Sorting Initialization:** Initialize an empty list L to store the sorted
543 OCL statements. Initialize an empty set S to store all nodes with no incoming edges.

544 **Identify Nodes with No Incoming Edges:** For each node $v \in V$, if v has no
545 incoming edges, add v to S .

546 **Topological Sorting Process:** While S is not empty: - Remove a node n from
547 S . - Add n to L . - For each node m such that $(n, m) \in E$: - Remove edge (n, m) from
548 E . - If m has no other incoming edges, add m to S .

549 **Cycle Detection:** If there are any remaining edges in E , this indicates that the
550 graph G contains at least one cycle, and a topological sorting is not possible. In this
551 case, report an error.

552 **Return the Sorted List:** Return the list L as the topologically sorted order of
553 OCL statements.

Algorithm 2 OCL Statement Execution Order Sorting

Input: A set of OCL statements $S = \{s_1, s_2, \dots, s_n\}$ with dependencies

Output: A topologically sorted list of OCL statements

```
1: Initialize an empty dependency graph  $G = (V, E)$ 
2: for each  $s_i \in S$  do
3:   Add  $s_i$  as a node in  $V$ 
4: end for
5: for each  $s_i \in S$  do
6:   for each  $s_j \in \text{dependencies}(s_i)$  do
7:     Add a directed edge  $(s_j, s_i)$  to  $E$ 
8:   end for
9: end for
10: Initialize an empty list  $L$  to store the sorted OCL statements
11: Initialize an empty queue  $Q$  to store all nodes with no incoming edges
12: for each  $v \in V$  do
13:   if  $v$  has no incoming edges then
14:     Add  $v$  to  $Q$ 
15:   end if
16: end for
17: while  $Q$  is not empty do
18:   Remove a node  $n$  from  $Q$ 
19:   Add  $n$  to  $L$ 
20:   for each node  $m$  such that  $(n, m) \in E$  do
21:     Remove edge  $(n, m)$  from  $E$ 
22:     if  $m$  has no other incoming edges then
23:       Add  $m$  to  $Q$ 
24:     end if
25:   end for
26: end while
27: if any edges remain in  $E$  then
28:   error: Graph  $G$  has at least one cycle
29: end if
30: return  $L$ 
```

4.3 Quality Assessment and Refactoring Recommendations of Design Model

We evaluated design quality using QMOOD, a hierarchical model to evaluate object-oriented design quality. In the QMOOD method, the high-level quality attribute indicators are calculated from the low-level element statistical indicators. Traditionally, this quality assessment is based on code. However, we noticed that many of these underlying metrics were available early in the design phase of the project, empowering designers to make proactive decisions. For example, QMOOD uses the number of classes to represent the size of the design, which is determined when the design

563 is completed. Another example is the number of levels in a hierarchical architecture,
564 whose value is fixed and can also be obtained at design time.

565 4.3.1 Rule-based Design refactoring Agent

566 Based on the results of the quality attributes assessment, we provide simple rule-based
567 design refactoring recommendations to designers. Since the calculation of high-level
568 quality attribute indicators is based on the statistical number of underlying elements,
569 we can identify the design elements that most impact a particular quality attribute
570 indicator. We decided to set the granularity of design elements to classes, and when
571 a class was found to have the most significant negative impact on specific quality
572 attribute metrics, we highlighted it and suggested that designers make refactorings.

- 573 • Design size: Consider the case where the number of classes exceeds a threshold
574 that indicates complexity. This metric is available early in the design phase. If the
575 number of classes is too high, we recommend strategies such as refactoring into
576 smaller modules or applying design patterns like the Factory pattern to manage
577 complexity.
- 578 • Hierarchy: In a hierarchical architecture, if the number of levels is too deep, indi-
579 cating a potential for maintenance issues, we suggest flattening the structure by
580 merging similar classes or using composition over inheritance where appropriate.
- 581 • Coupling: When the coupling between classes is too high, indicating a lack of mod-
582 ularity, we recommend introducing interfaces or applying the Dependency Injection
583 pattern to decouple components.
- 584 • Cohesion: If the cohesion within a class is low, suggesting that the class has multiple
585 responsibilities, we advise splitting the class into smaller, more focused classes or
586 applying the Single Responsibility Principle.
- 587 • Polymorphism: If there is a high degree of polymorphism, indicating a com-
588 plex inheritance hierarchy, we recommend simplifying the inheritance structure by
589 reducing the depth of the hierarchy or employing composition instead of inheritance.

590 For example, in the generated class diagram shown in Figure 2, we can see that
591 the EntityManager class has the most significant number of directly related classes,
592 which has the highest degree of coupling according to the metric calculation, thus
593 suggesting a design refactoring for it: “EntityManager has high Direct Class Cou-
594 pling, please pay attention to proper class responsibility division”. Considering the
595 refactoring of EntityManager, in this design, EntityManager is mainly responsible for
596 receiving the requests of the service layer and modifying the attributes and associ-
597 ation relations of the related classes in the entity layer. The EntityManager can be
598 split into multiple repositories, each responsible for modifying a class. This reduces
599 the degree of coupling between classes, and the responsibility of each class is more sin-
600 gle, in line with the single responsibility principle. We can see that this modification
601 has changed some of the low-level design metrics and affected the high-level design
602 quality. Understandability decreases because multiple managers are more complex
603 than a single Manager in terms of design structure. However, splitting it into various
604 managers makes for better readability. Functionality goes up because multiple Man-
605 agers bring access to multiple common interfaces. Less coupling between classes also

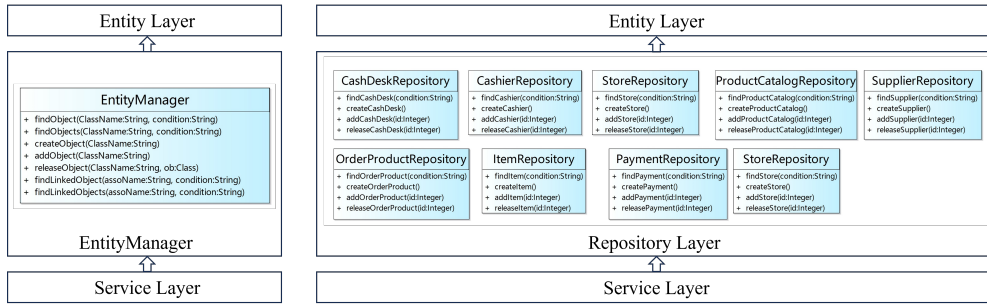
Table 1 Comparison of Design Quality Indicators Before and After Modification

Metric	DSC	NOH	ANA	DAM	DCC	CAM	MOA	MFA	NOP	CIS	NOM
Before	24.0000	3.0000	0.0833	1.0000	2.7917	1.0000	5.0000	0.4500	1.0000	144.0000	144.0000
After	32.0000	3.0000	0.0625	1.0000	2.0625	1.0000	5.0000	0.4500	1.0000	173.0000	173.0000
After/Before	1.3333	1.0000	0.7503	1.0000	0.7388	1.0000	1.0000	1.0000	1.0000	1.2014	1.2014

Table 2 Comparison of Design Quality Before and After Modification

Metric	Resuability	Flexibility	Understandability	Functionality	Extendibility	Effectiveness
Before	1.0000	1.0000	-0.9900	1.0000	1.0000	1.0000
After	1.3327	1.0653	-0.9979	1.1176	1.0058	0.9501

leads to better reusability, flexibility, and extensibility. This example illustrates the effectiveness of the refactoring method according to quality metrics.

**Fig. 6** Optimized Class Diagram

4.3.2 LLM-based Design refactoring Agent

The rapid development of Large Language Models (LLMs), such as GPT, BERT, and their successors, has revolutionized various domains by providing unprecedented capabilities in natural language understanding and generation. These models are not only reshaping industries like customer support, content creation, and data analytics but are also increasingly being leveraged in specialized technical domains. In the realm of Object-Oriented (OO) design refactoring, LLMs have demonstrated their potential to analyze complex software models, identify design inefficiencies, and suggest improvements grounded in best practices and design principles such as SOLID and GRASP. By integrating domain-specific knowledge with pattern recognition, LLMs provide actionable recommendations, enhance maintainability, and reduce technical debt, making them a valuable tool for modern software engineering. In our approach, GPT-based capabilities are utilized in three key aspects:

- Providing design refactoring recommendations based on the current quality metrics.

Model refactoring Suggestion

PROMPT

The plantUML of Design is as follows: [**DesignUML**]. The design quality of this design are as follows: [**DesignQuality**].

Based on design patterns (e.g., Singleton, Factory, Observer, etc.) and GRASP design principles (e.g., Information Expert, Low Coupling, High Cohesion), suggest improvements for the model. Please explain the following:

- 1. Which classes or relationships can be optimized? Provide specific suggestions for changes.
- 2. Based on design patterns, which patterns could enhance the maintainability, scalability, or performance of the model?
- 3. How can GRASP principles optimize the assignment of responsibilities and interactions between classes?
- 4. How will the proposed changes address potential issues in the existing model (e.g., high coupling, low cohesion)?
- 5. After modifications, which aspects of the model (e.g., class responsibilities, relationships, coupling) will improve?

Please analyze each suggestion step by step (Chain of Thought), providing an explanation for each and proposing the most suitable solution.

The overall structure should be that the first part is a series of specific change suggestions, and the second part is explanations of the change suggestions.

VARIABLE

[**DesignUML**]: The plantUML representation of Design Model.

[**DesignQuality**]: Design quality assessment results automatically computed by the modeling tool, employing the QMOOD metrics, including reusability, flexibility, understandability, functionality, extendibility and effectiveness.

622

623 The provided prompt is designed to facilitate comprehensive and structured analysis
624 of software design models, leveraging the capabilities of Large Language Models
625 (LLMs) to optimize Object-Oriented (OO) design. It integrates principles from
626 design patterns and GRASP (General Responsibility Assignment Software Patterns)
627 to ensure that recommendations align with established best practices in
628 software engineering. By using a PlantUML representation of the design model
629 as input, the prompt focuses on actionable improvements, making it suitable for
630 iterative development processes.

631 One of the key strengths of this prompt lies in its focus on clarity and step-by-
632 step reasoning. The Chain of Thought (CoT) methodology is explicitly embedded,
633 encouraging the model to break down complex design quality assessments into man-
634 ageable components. This approach not only improves the interpretability of the
635 recommendations but also ensures that the reasoning behind each suggestion is
636 transparent and traceable. Furthermore, the prompt emphasizes a dual-structured

637 response, first presenting concrete recommendations and then providing detailed
 638 justifications. This structure fosters a deeper understanding of how the proposed
 639 changes will improve design quality metrics, such as maintainability, scalability,
 640 and performance.

641 Another notable advantage is the explicit integration of both design patterns and
 642 GRASP principles. By asking the model to evaluate the applicability of patterns
 643 such as Singleton, Factory, and Observer, the prompt ensures that the recommen-
 644 dations are grounded in established strategies for solving common design challenges.
 645 Simultaneously, GRASP principles guide the analysis of responsibility allocation
 646 and interactions between classes, addressing issues like high coupling and low cohe-
 647 sion. This dual focus enables a holistic quality assessment of the design, balancing
 648 the structural and behavioral aspects of the model.

- 649 • Automatically generating explanations for changes in design quality metrics follow-
 650 ing specific operations.

Model refactoring Explanation

PROMPT

The plantUML of Design is as follows: **[DesignUML]**. Based on the modified UML Class Diagram, explain why these changes will impact the quality of the model. Specifically, explain how the following aspects are improved or degraded:

- 1. Has the distribution of responsibilities among classes become more reasonable? Has cohesion been improved?
- 2. Has the coupling between classes been effectively reduced after the modifications?
- 3. How do the newly added design patterns or GRASP principles affect the scalability, maintainability, or performance of the model?
- 4. Has the design reduced complexity and improved the understandability of the model?
- 5. Overall, have the quality metrics of the model (e.g., flexibility, reusability) been improved? If so, in which specific areas?

Please explain step by step (Chain of Thought) the specific impact of each change.

VARIABLE

[DesignUML]: The plantUML representation of Design Model.

651 The accompanying prompt is crafted to enable detailed, step-by-step explanations
 652 of the impacts of design modifications on software quality metrics. It provides
 653 a structured framework that leverages the analytical capabilities of Large Lan-
 654 guage Models (LLMs) to assess how specific changes in a UML Class Diagram
 655 influence the overall design quality. By embedding Chain of Thought (CoT) rea-
 656 soning, the prompt ensures that each modification is analyzed comprehensively and
 657

methodically, making it a powerful tool for understanding and refining software models.

A core strength of the prompt lies in its focus on critical aspects of design evaluation, such as cohesion, coupling, scalability, and maintainability. These are foundational metrics in assessing software quality, and the prompt explicitly calls for a systematic analysis of how each metric is impacted by the proposed changes. For example, by addressing whether responsibility distribution has become more logical or whether cohesion has improved, the prompt ensures that the analysis is grounded in principles of Object-Oriented Design (OOD). Similarly, the explicit focus on coupling reduction and the application of design patterns or GRASP principles highlights the importance of achieving balance in structural and behavioral design.

Moreover, the prompt emphasizes actionable insights by asking how design patterns and GRASP principles enhance the model's scalability, maintainability, and performance. This ensures that the explanations are not only theoretical but also practical, providing developers with a clear understanding of the trade-offs and benefits introduced by specific design decisions. The inclusion of considerations like complexity reduction and understandability further aligns the prompt with real-world development goals, ensuring that the resulting models are both robust and developer-friendly.

- Analyzing the traceability between the current design and specific requirements, identifying unmet requirements, and suggesting complementary design elements to address the gaps. The prompt is expertly designed to facilitate a comprehensive and structured analysis of traceability between design and requirements in software engineering. It leverages the capabilities of Large Language Models (LLMs) to ensure precise mapping of requirements to design elements while identifying gaps and proposing actionable solutions. The use of a PlantUML-based representation for the design and a detailed requirements file ensures that the analysis is grounded in clear, visual, and textual artifacts, enhancing the clarity and accuracy of the recommendations.

A particularly valuable aspect of the prompt is its focus on unmet requirements. By explicitly asking the model to identify and explain why certain requirements are missing or overlooked, it fosters a deeper understanding of potential gaps in the design process. This ensures that the analysis not only addresses technical shortcomings but also considers contextual factors such as oversight, complexity, or misalignment of goals.

The recommendation for complementary design elements further enhances the prompt's utility. By requiring the model to propose solutions aligned with established best practices like SOLID principles, the prompt ensures that the suggestions are both practical and theoretically sound. This dual focus on maintainability and scalability reflects a strong alignment with modern software engineering priorities, making the output highly actionable for practitioners.

The final step, justification and validation, adds an extra layer of rigor to the analysis. By asking the model to explain the reasoning behind proposed solutions and discuss potential trade-offs, the prompt ensures that the recommendations are balanced and well-considered. This not only aids in decision-making but also provides

703 valuable insights into the implications of the proposed changes, such as their impact
704 on design quality metrics like cohesion, coupling, and reusability.

Traceability Between Design and Requirements

PROMPT

The plantUML of Design is as follows: [DesignUML]. The Requirements is as follows: [Remodel].

I am designing a software system and need your expertise in analyzing the traceability between the current design and specific requirements. Please follow these steps to provide a detailed and actionable response:

- 1. Traceability Analysis: Begin by identifying the key requirements from the provided list. Map each requirement to the corresponding design elements.
- 2. Unmet Requirements Identification: Highlight any requirements that are not addressed by the current design. For each unmet requirement, explain why it might be missing or overlooked.
- 3. Recommendations for Complementary Design Elements: For each unmet requirement, propose specific design elements or modifications to the existing design. Ensure that the recommendations align with best practices in software design (e.g., SOLID principles, maintainability, scalability).
- 4. Justification and Validation: Provide reasoning for the proposed design elements, explaining how they fulfill the unmet requirements. Discuss any potential trade-offs or impacts on the overall design quality metrics.

Use a structured format for your response, such as:

Requirement X: (State the unmet requirement)

Current Coverage: (Briefly describe if it's partially covered or missing entirely)

Proposed Solution: (Detail the complementary design element)

Justification: (Explain why this solution is appropriate and its potential effects)

Finally, ensure that your response is concise yet comprehensive, avoiding unnecessary jargon. If additional clarification or details about the current design are needed, specify the information required.

VARIABLE

[plantUML]: The plantUML representation of Design Model.

[Remodel]: The Remodel file of Requirements.

705

706 5 Evaluation

707 In this section, We use six requirement models of different software systems to perform
708 evaluation.

709 5.1 Research Questions

- 710 • **RQ1:** To what extent do the transformation rules proposed by our approach cover
711 various situations?
- 712 • **RQ2:** How is the quality of the generated design models, and how does it compare
713 to manually created models?
- 714 • **RQ3:** Does our automated approach for constructing design models offer a time
715 efficiency advantage over manual approaches?.

716 5.2 Experimental Setting

717 We developed a tool based on the Eclipse MDA framework to automate our approach.
718 We use ATL transformation language to implement the requirements model to design
719 model transformation and Sirius to graphically display and modify the design model.
720 The tool provides the functionality to edit the generated design models. During the
721 design model generation and modification process, automatic metrics calculation and
722 identification of design elements to be optimized are achieved by writing Java Services
723 for Sirius.

724 To address RQ1, we start by counting the relevant measures within both the
725 requirement model and the automatically generated design model. These counts are
726 then used to calculate specific metrics that are crucial for our analysis.

727 For RQ2, we enlist the expertise of seasoned professionals in the software engi-
728 neering field. These experts are carefully recruited based on their extensive experience
729 and in-depth knowledge of design models and domain modeling. They are tasked with
730 establishing design models that conform to an anaemic domain model, derived directly
731 from the requirement models. These expert-generated models serve as the reference
732 models for our study. To facilitate comparison, we also involve postgraduate students
733 who are specializing in software engineering. These students are recruited through
734 academic channels, focusing on those who have demonstrated proficiency in software
735 design and development in their coursework.

736 Before the postgraduates begin constructing the design models, they undergo a
737 comprehensive training program. This program is structured to ensure they have a
738 thorough understanding of the tools and methodologies they will be using, as well
739 as the concepts of requirement modeling and anaemic domain models. The training
740 involves hands-on sessions where the participants are introduced to the tools necessary
741 for constructing design models. They also receive detailed explanations of the models
742 they are expected to build, focusing particularly on the principles of the anaemic
743 domain model and how it aligns with the requirement model.

744 Once the training is complete, the participants are provided with a specific set
745 of requirement documents. Their task is to construct a design model based on these
746 documents, adhering to the principles of the anaemic domain model. The design model
747 they are asked to create is for a monolithic system built using SpringBoot. This part of
748 the experiment is critical as it allows us to directly compare the manually constructed
749 design models with those generated automatically, thereby providing insights into the
750 effectiveness and efficiency of each approach.

For RQ3, we focus on the time efficiency of the design model construction process. We meticulously record the time taken by the postgraduates to manually construct the design model. This duration is then compared with the time required to build the model using our automated method. The comparison of these time metrics provides valuable data on the efficiency gains that our automated approach offers over traditional manual methods.

5.3 Case Studies

The dataset has six requirement models, four of which come from RM2PT. The cases of the six requirement models describe the requirements of six common scenarios, including automatic teller machine(ATM), supermarket system(CoCoME), library management system(LibMS), loan management system(LoanPS), Parking spot share system(ParkShare) and Airport management system(AirMS). The case of an automatic teller machine mainly involves the user's need to use an ATM to deposit and withdraw money. It is relatively simple and can be used as a rapid preliminary verification of the automatic generation method of the design model. The case of CoCoME describes the needs of the cashier in the supermarket to process the shopping documents and the administrator to manage the supermarket inventory. The process involved in this case is complex, and it can be verified whether the generated model can describe the interaction between the objects in the system well. The case of LibMS describes the need for students to borrow and return books. In this case, the system operation is complex, which can verify whether the generated design model can encapsulate the system operation well. The case of LoanPS describes the user's demand for loans. To fulfill this demand, it is necessary to call the services of the third party, which can verify whether the generated design model can well describe the scenario where the system interacts with other systems to complete functions. The ParkShare case allows people to rent or share their available parking spaces, which are relatively small in scale. The AirMS case addresses the needs of airport equipment management and has a larger scale. Both case studies are derived from real systems and have requirement models constructed by experienced developers. The complexity of the six requirement models is close to the complexity of the enterprise monolithic system that this study is aimed at, and it is suitable for the data set of this experiment. Make statistics on the number of model elements in the requirement model, and the statistical results are shown in Table 3.

5.4 Evaluation Metrics

5.4.1 Transformation Rule Metrics

These metrics evaluate the extent to which the proposed transformation rules cover the transformation from requirements models to design models.

- **Success rate of entity-to-class transformation.** $CSRate = RC/AC$, where RC denotes the number of correct entity classes in the class diagram, and AC denotes the number of entities in the conceptual class diagram.

Table 3 The Complexity of Requirements Models

Case Study	Actor	Use Case	SO	OCL	Entity Class	Association
ATM	2	6	15	65	3	4
CoCoME	3	16	43	198	13	20
LibMS	7	19	45	303	11	17
LoanPS	5	10	34	127	12	8
ParkShare	4	7	12	48	7	11
AirMS	6	33	58	336	15	19
Sum	27	91	207	1077	61	79

* SO is the abbreviations of system operations. OCL refers to the number of OCL statements that can be converted into messages in the sequence diagram.

- **Success rate of association-to-relationship transformation.** $ASRate = RA/AA$, where RA denotes the number of correct relationships between entity classes in the class diagram, and AA denotes the number of associations in the conceptual class diagram.
- **Correctness of encapsulation of operations.** $OSRate = RO/AO$, where RO indicates the number of system operations in the class diagram that are correctly encapsulated into interface classes, and AO indicates the total number of system operations in contracts.
- **Success rate of message generation in sequence diagram.** $MSRate = RM/AM$, where AM refers to the number of OCL expressions to be converted, and RM refers to the number of messages correctly converted from OCL expressions. Similarly, we can calculate $Def - MSRate$ and $Post - MSRate$ for the contract's definition and postcondition parts, respectively.

5.4.2 Design Quality Metrics

We evaluate design quality using high-level quality attribute metrics in QMOOD. We take the quality attributes of design models manually constructed by experts as the basis and normalize the quality attribute indicators of automatic and semi-automatic generated design models with a few fast iterations for comparison.

For the sake of descriptive clarity, formulas for calculating the high-level design metrics in QMOOD are provided.

$$Reusability = -0.25 * DCC + 0.25 * CAM + 0.5 * CIS + 0.5 * DSC$$

$$Flexibility = 0.25 * DAM - 0.25 * DCC + 0.5 * MOA + 0.5 * NOP$$

$$Understandability = -0.33 * ANA + 0.33 * DAM - 0.33 * DCC \\ + 0.33 * CAM - 0.33 * NOP - 0.33 * NOM - 0.33 * DSC$$

$$Functionality = 0.12 * CAM + 0.22 * NOP + 0.22 * CIS + 0.22 * DSC + 0.22 * NOH$$

813

$$Extendibility = 0.5 * ANA - 0.5 * DCC + 0.5 * MFA + 0.5 * NOP$$

814

$$Effectiveness = 0.2 * ANA + 0.2 * DAM + 0.2 * MOA + 0.2 * MFA + 0.2 * NOP$$

815 The meanings of the variables on the right-hand side of the equation are shown in the
816 following Table 4.

Table 4 Meaning of Variables

Variables	Meaning
DSC	Design Size in Classes
NOH	Number of Hierarchies
ANA	Average Number of Ancestors
DAM	Data Access Metric
MFM	Measure of Functional Modularity
DCC	Direct Class Coupling
CAM	Cohesion Among Methods in Class
MOA	Measure of Aggregation
MFA	Measure of Function Abstraction
NOP	Number of Polymorphic Methods
CIS	Class Interface Size
NOM	Number of Methods

817 5.5 Evaluation Results

818 5.5.1 RQ1: To what extent do the transformation rules proposed by 819 our approach cover various situations

820 Our tool can successfully and correctly generate all class diagrams of these six case
821 studies. The results of the sequence diagram generation are shown in Table 5. On
822 average, 98.50% (DSRate) of the expressions in the definition part of the contract
823 is correctly transformed into messages in the sequence diagram, 93.96% (PSRate) in
824 the post-condition part, and 94.80% (MSRate) of the overall messages are correctly
825 generated. This result shows that our proposed rule can completely cover the case
826 of class diagram generation and can basically cover the case of sequence diagram
827 generation.

828 After analysis, the failure to generate messages includes the following:

- 829 1. The developed tool cannot handle multiple nested If-else statements.
- 830 2. The contract statement is wrongly written.
- 831 3. The return value of the contract is not a variable, but an OCL operation statement.

Table 5 The Generation Result of Sequence Diagram Message

Case Study	Def-OCL	Def-Message	DSRate	Post-OCL	Post-Message	PSRate	MSRate
ATM	9	9	100.00%	56	48	85.71%	87.69%
CoCoME	35	34	97.14%	163	156	95.71%	95.71%
LibMS	59	57	96.61%	244	223	91.39%	92.41%
LoanPS	19	19	100.00%	108	104	96.30%	96.85%
ParkShare	5	5	100.00%	43	42	97.67%	97.92%
AirMS	73	73	100.00 %	263	251	95.44%	96.43%
Sum	200	197	98.50%	877	824	93.96%	94.80%

* Def-OCL refers to the OCL expressions to be converted in the definition part of the system operation contract, and Def-Message refers to the number of messages correctly converted from Def-OCL; Post-OCL refers to the OCL expressions to be converted in the post-condition part of the system operation contract, and Post-Message refers to the number of messages correctly converted from Post-OCL.

5.5.2 RQ2: How is the quality of the generated design models, and how does it compare to manually created models

In our study, we used the design model that conforms to the anemic domain model constructed by experts as the correct reference model. Based on this reference model, we calculated the model quality metrics for design models manually constructed by graduate students in the software engineering field and those automatically generated by our tool, as shown in Table 6. The arithmetic mean calculation based on six experimental datasets reveals that design models generated by RapidOOD achieve 89.20% (without human intervention) and 94.56% (under human-in-the-loop collaboration) of the quality assessment metrics attained by expert-developed models, respectively.

We also compared the performance of our method with that of directly using large language models (LLMs), including single LLM and multi LLMs. For single LLMs, we provided the requirement model to the LLM and requested it to directly generate the corresponding design model. As for multi LLMs, we created a workflow that includes three LLM nodes. One was responsible for design generation, another for quality assessment, and the third for refactoring. The process of generation-assessment-refactoring was iterated five times, after which the final result was output.

According to the findings, we can conclude that:

1. The initial designs generated by our method can achieve a quality level close to that of experts. Moreover, with the assistance of software, after a few manual adjustments, the designs can reach the level of experts in some indicators, and overall, attain the same quality standard as that of experts.
2. Compared with methods that merely rely on LLMs, our approach exhibits substantial advantages in quality metrics. Through analysis, the primary reason is that the scale of results output by LLMs is limited, and this limited scale confers better understandability. In terms of indicators such as Reusability, Flexibility, Functionality, Extendibility, and Effectiveness, methods solely using LLMs can hardly reach a level comparable to ours. This reflects the benefits of integrating the model-driven approach with LLMs. Employing a rule-based method to generate the initial design can: 1) Ensure the initial scale of the design model; 2) Guarantee

Table 6 Comparison of Results: Our Automated Approach and LLM-Only Application

Case Study	Quality	LLM Only		RapidOOD(LLM+Rule)	
		Single LLM	Multi LLM	Without Human	Human in the Loop
ATM	Reusability	0.49	0.90	0.87	1
	Flexibility	0.15	0.24	0.8	1
	Understandability	-0.21	-1.16	-0.93	-1
	Funcationality	0.44	0.49	0.98	0.98
	Extendibility	0.50	0.42	1	1
	Effectiveness	0.32	0.34	0.99	0.99
Case Study	Quality	Single LLM	Multi LLM	Without Human	Human in the Loop
CoCoME	Reusability	0.38	0.47	0.86	1
	Flexibility	0.18	0.23	0.98	1
	Understandability	-0.02	-0.15	-0.98	-1
	Funcationality	0.37	0.29	0.96	0.96
	Extendibility	0.25	0.47	0.86	1
	Effectiveness	0.39	0.37	0.93	0.98
Case Study	Quality	Single LLM	Multi LLM	Without Human	Human in the Loop
LibraryMS	Reusability	0.34	0.42	0.90	1
	Flexibility	0.32	0.37	0.98	1
	Understandability	-0.24	-0.12	-0.97	-1
	Funcationality	0.35	0.30	0.99	0.99
	Extendibility	0.32	0.33	0.89	0.97
	Effectiveness	0.36	0.43	0.96	0.98
Case Study	Quality	Single LLM	Multi LLM	Without Human	Human in the Loop
LoanPS	Reusability	0.51	0.56	0.89	0.98
	Flexibility	0.21	0.08	0.96	1
	Understandability	-0.42	-0.52	-0.98	-1
	Funcationality	0.39	0.48	0.99	0.99
	Extendibility	0.42	0.48	0.94	1
	Effectiveness	0.34	0.26	0.97	1
Case Study	Quality	Single LLM	Multi LLM	Without Human	Human in the Loop
ParkShare	Reusability	0.32	0.66	0.96	1
	Flexibility	0.01	0.14	0.92	1
	Understandability	-0.05	-0.21	-0.97	-1
	Funcationality	0.22	0.38	0.98	0.98
	Extendibility	0.50	0.48	0.93	1
	Effectiveness	0.20	0.30	0.97	0.99
Case Study	Quality	Single LLM	Multi LLM	Without Human	Human in the Loop
AirMS	Reusability	0.55	0.28	0.87	0.99
	Flexibility	0.23	0.20	0.90	0.97
	Understandability	-0.32	-0.17	-0.93	-0.99
	Funcationality	0.64	0.36	0.93	0.96
	Extendibility	0.39	0.44	0.95	0.96
	Effectiveness	0.33	0.34	0.95	0.96

¹ We adopt the design model manually constructed by experts based on the requirements model as the benchmark. The results in the table are the ratios of quality attributes compared to the benchmark.

² *LLM Only* column: results obtained solely by posing questions to the LLM. *Single LLM*: Requirements model provided to LLM for direct design model generation. *Multi LLM*: Design model from 5 “Generate - Evaluate - Optimize” iteration with 3 LLM nodes.

³ *RapidOOD* column: *Without Human* - initial design by Design Generation Agent only; *Human in the Loop* - initial design adjusted by human per evaluation and refactoring suggestions after a few iterations.

863 the traceability between the design model and the requirement model; 3) Prevent
864 the introduction of hallucinations from the output of LLMs.

5.5.3 RQ3: Does our automated approach for constructing design models offer a time efficiency advantage over manual approaches

In the process of manually constructing the design model, we provided the requirement model, which typically includes use case diagrams, class diagrams, and system sequence diagrams. In our approach, we additionally employed OCL contracts to constrain system operations. The time spent on writing OCL contracts constitutes the primary time consumption of our automated method. Therefore, we compared the time consumed by OCL contracts with that of manually constructing design models, as shown in Table 2. As can be seen, the average time taken to construct OCL contracts is 4.6 hours, while the average time for manually constructing design models is 6.4 hours. Our approach can save 28.1% of the time consumption, achieving 1.39 times the efficiency of the manual method.

Table 7 Time Consuming of Automated Mean and Manual Mean

Case Study	OCL Cost(h)	Manual Mean(h)	Time Saving Rate
ATM	1.3	2.0	35.00%
CoCoME	4.9	7.5	34.67%
LibMS	6.4	8.3	22.89%
LoanPS	6.9	7.2	4.17%
ParkShare	3.1	4.2	26.19%
AirMS	5.0	9.2	45.65%
Average	4.6	6.4	28.09%

5.6 Discussion and Limitation

Using lightweight formal requirement models as input, our approach can successfully generate design models that conform to the anemic domain model. Our approach outperforms graduate students in the software engineering field in terms of completeness and consistency with expert-constructed design models. However, our method has the following limitations.

Our approach does not currently support the handling of multiple nested logical judgments. This limitation restricts the ability of our tool to capture complex relationships and constraints within the requirements. Nested logical judgments often occur in real-world systems, especially in those with intricate business rules. Therefore, extending our approach to support such constructs would significantly enhance its applicability and robustness.

Our method relies on users to correctly write OCL (Object Constraint Language) contracts. While OCL provides a powerful means to express constraints, it is a specialized language that may not be familiar to all stakeholders involved in the software development process. Thus, the need for expertise in OCL might limit the usability

894 of our approach, particularly for non-technical stakeholders. Providing better sup-
895 port for OCL validation or integrating a user-friendly interface for generating OCL
896 contracts could help mitigate this issue.

897 Currently, our approach only generates anemic domain model designs and cannot
898 support the generation of a wider variety of design models, such as rich domain models
899 or domain-driven design patterns. Anemic domain models, while useful, do not always
900 capture the full richness of the domain logic. Expanding our approach to support
901 other design patterns could improve the expressiveness and flexibility of the generated
902 design models.

903 5.7 Threats to Validity

904 There are several threats to the validity of our study.

905 The dataset used in this experiment consists of only six requirement models, limit-
906 ing the generalizability of our findings. The automatic generation of the class diagram
907 in the design model is only conducted six times, and the evaluation of the success
908 rate of the automatic generation of the class diagram may not be sufficient to fully
909 validate the effectiveness of our approach across different domains and complexities.

910 The complexity of the data centralized system used in this experiment is close to
911 that of a single enterprise information system. However, there are still differences in
912 the actual business processes and requirements, which may affect the rationality of
913 the experimental evaluation. To address this, future work should include experiments
914 with a broader range of systems and varying levels of complexity.

915 We use expert-constructed design models as the correct answers for comparison.
916 While this approach is common, it may introduce human bias. Expert opinions can
917 vary, and there may not be a single correct design solution. To mitigate this threat,
918 we could conduct additional studies involving multiple experts and consensus-building
919 processes to ensure the reliability of the baseline models.

920 6 Conclusion and Future Work

921 This paper proposes the RapidOOD approach to automated design model gener-
922 ation from lightweight formal requirements models. Work like RM2PT can help
923 achieve a validated requirements model by automatically generating prototypes from
924 requirements models. RapidOOD generates OO design models from this validated
925 requirements model, further alleviating the software development problem. The
926 generated design models can be used as references for designers and shorten the time-
927 consuming design phase. Six case studies evaluated the capabilities of RapidOOD. The
928 experiment result is satisfactory and shows that the approach is effective and efficient.

929 In the future, We will consider combining the anemic model and domain model
930 design philosophies to guide the automated generation of designs, thus addressing the
931 automatic generation of design models for a broader range of systems.

932 **Acknowledgements.** This work was supported by the National Natural Science
933 Foundation of China (No.62302029), State Key Laboratory of Complex & Critical
934 Software Environment.

References

- [1] Eden, A.H., Kazman, R.: Architecture, design, implementation. In: 25th International Conference on Software Engineering, 2003. Proceedings., pp. 149–159 (2003). <https://doi.org/10.1109/ICSE.2003.1201196>
- [2] Flageol, W., Menaud, ., Guéhéneuc, Y.-G., Badri, M., Monnier, S.: A mapping study of language features improving object-oriented design patterns **160**, 107222 <https://doi.org/10.1016/j.infsof.2023.107222> . Accessed 2024-04-01
- [3] Li, X., Liu, Z., He, J.: Formal and use-case driven requirement analysis in uml. In: 25th Annual International Computer Software and Applications Conference. COMPSAC 2001, pp. 215–224 (2001). <https://doi.org/10.1109/COMPSAC.2001.960619>
- [4] Liu, Z., Jifeng, H., Li, X., Chen, Y.: A relational model for formal object-oriented requirement analysis in uml. In: Dong, J.S., Woodcock, J. (eds.) Formal Methods and Software Engineering, pp. 641–664. Springer, Berlin, Heidelberg (2003)
- [5] Chen, Z., Liu, Z., Ravn, A.P., Stolz, V., Zhan, N.: Refinement and verification in component-based model-driven design. Science of Computer Programming **74**(4), 168–196 (2009) <https://doi.org/10.1016/j.scico.2008.08.003> . Special Issue on the Grand Challenge
- [6] Yang, Y., Li, X., Liu, Z., Ke, W.: Rm2pt: A tool for automated prototype generation from requirements model. In: 2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion), pp. 59–62 (2019). <https://doi.org/10.1109/ICSE-Companion.2019.00038>
- [7] Yang, Y., Ke, W., Li, X.: Rm2pt: Requirements validation through automatic prototyping. In: 2019 IEEE 27th International Requirements Engineering Conference (RE), pp. 484–485 (2019). <https://doi.org/10.1109/RE.2019.00067>
- [8] Yang, Y., Li, X., Ke, W., Liu, Z.: Automated prototype generation from formal requirements model. IEEE Transactions on Reliability **69**(2), 632–656 (2020) <https://doi.org/10.1109/TR.2019.2934348>
- [9] Bao, T., Yang, J., Yang, Y., Yin, Y.: Rm2doc: a tool for automatic generation of requirements documents from requirements models. In: Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings. ICSE ’22, pp. 188–192. Association for Computing Machinery, New York, NY, USA (2022). <https://doi.org/10.1145/3510454.3516850>
- [10] Easterbrook, S., Lutz, R., Covington, R., Kelly, J., Ampo, Y., Hamilton, D.: Experiences using lightweight formal methods for requirements modeling. IEEE Transactions on Software Engineering **24**(1), 4–14 (1998) <https://doi.org/10.1109/32.663994>

- [11] Mala, G.S.A., Uma, G.V.: Automatic construction of object oriented design models [uml diagrams] from natural language requirements specification. In: Yang, Q., Webb, G. (eds.) PRICAI 2006: Trends in Artificial Intelligence, pp. 1155–1159. Springer, Berlin, Heidelberg (2006)
- [12] Sharma, V.S., Sarkar, S., Verma, K., Panayappan, A., Kass, A.: Extracting high-level functional design from software requirements. In: 2009 16th Asia-Pacific Software Engineering Conference, pp. 35–42 (2009). <https://doi.org/10.1109/APSEC.2009.63>
- [13] Yue, T., Briand, L.C., Labiche, Y.: atoucan: An automated framework to derive UML analysis models from use case models. *ACM Trans. Softw. Eng. Methodol.* **24**(3), 13–11352 (2015)
- [14] Easterbrook, S., Lutz, R., Covington, R., Kelly, J., Ampo, Y., Hamilton, D.: Experiences using lightweight formal methods for requirements modeling. *IEEE Transactions on Software Engineering* **24**(1), 4–14 (1998) <https://doi.org/10.1109/32.663994>
- [15] Li, X., Liu, Z., He, J.: Formal and use-case driven requirement analysis in uml. In: 25th Annual International Computer Software and Applications Conference. COMPSAC 2001, pp. 215–224 (2001). <https://doi.org/10.1109/CMPSAC.2001.960619>
- [16] Chen, Z., Hannousse, A.H., Van Hung, D., Knoll, I., Li, X., Liu, Z., Liu, Y., Nan, Q., Okika, J.C., Ravn, A.P., Stolz, V., Yang, L., Zhan, N.: In: Rausch, A., Reussner, R., Mirandola, R., Plášil, F. (eds.) *Modelling with Relational Calculus of Object and Component Systems - rCOS*, pp. 116–145. Springer, Berlin, Heidelberg (2008). https://doi.org/10.1007/978-3-540-85289-6_6
- [17] Chen, Z., Liu, Z., Ravn, A.P., Stolz, V., Zhan, N.: Refinement and verification in component-based model-driven design. *Science of Computer Programming* **74**(4), 168–196 (2009) <https://doi.org/10.1016/j.scico.2008.08.003> . Special Issue on the Grand Challenge
- [18] Liu, Z., Morisset, C., Stolz, V.: rcos: Theory and tool for component-based model driven development. In: Arbab, F., Sirjani, M. (eds.) *Fundamentals of Software Engineering*, pp. 62–80. Springer, Berlin, Heidelberg (2010)
- [19] Koch, N., Zhang, G., Escalona, M.J.: Model transformations from requirements to web system design. In: *Proceedings of the 6th International Conference on Web Engineering. ICWE '06*, pp. 281–288. Association for Computing Machinery, New York, NY, USA (2006). <https://doi.org/10.1145/1145581.1145637> . <https://doi.org/10.1145/1145581.1145637>
- [20] Mussbacher, G., Kienzle, J., Amyot, D.: Transformation of aspect-oriented

- requirements specifications for reactive systems into aspect-oriented design specifications. In: 2011 Model-Driven Requirements Engineering Workshop, pp. 39–47 (2011). <https://doi.org/10.1109/MoDRE.2011.6045365>
- [21] Chidamber, S.R., Kemerer, C.F.: A metrics suite for object oriented design. IEEE Transactions on Software Engineering **20**(6), 476–493 (1994) <https://doi.org/10.1109/32.295895>
- [22] Lorenz, M., Kidd, J.: Object-oriented Software Metrics: a Practical Guide. Prentice-Hall, Inc., USA (1994)
- [23] Abreu, F.B., Carapuça, R.: Object-oriented software engineering: Measuring and controlling the development process. In: Proceedings of the 4th International Conference on Software Quality, vol. 186, pp. 1–8 (1994)
- [24] Bansiya, J., Davis, C.G.: A hierarchical model for object-oriented design quality assessment. IEEE Transactions on Software Engineering **28**(1), 4–17 (2002) <https://doi.org/10.1109/32.979986>
- [25] Pham, V., Lokan, C., Kasmarik, K.: A better set of object-oriented design metrics for within-project defect prediction. In: Proceedings of the 24th International Conference on Evaluation and Assessment in Software Engineering. EASE '20, pp. 230–239. Association for Computing Machinery, New York, NY, USA (2020). <https://doi.org/10.1145/3383219.3383243> . <https://doi.org/10.1145/3383219.3383243>
- [26] Yan, M., Xia, X., Zhang, X., Xu, L., Yang, D.: A systematic mapping study of quality assessment models for software products. In: 2017 International Conference on Software Analysis, Testing and Evolution (SATE), pp. 63–71 (2017). <https://doi.org/10.1109/SATE.2017.16>
- [27] Insfran, E., Gonzalez-Huerta, J., Abrahão, S.: Design guidelines for the development of quality-driven model transformations. In: Petriu, D.C., Rouquette, N., Haugen, Ø. (eds.) Model Driven Engineering Languages and Systems, pp. 288–302. Springer, Berlin, Heidelberg (2010)
- [28] Rähä, O., Koskimies, K., Mäkinen, E.: Generating software architecture spectrum with multi-objective genetic algorithms. In: 2011 Third World Congress on Nature and Biologically Inspired Computing, pp. 29–36 (2011). <https://doi.org/10.1109/NaBIC.2011.6089413>
- [29] Bowman, M., Briand, L.C., Labiche, Y.: Solving the class responsibility assignment problem in object-oriented analysis with multi-objective genetic algorithms. IEEE Transactions on Software Engineering **36**(6), 817–837 (2010) <https://doi.org/10.1109/TSE.2010.70>
- [30] Hadaytullah, Vathsavayi, S., Rähä, O., Koskimies, K.: Tool support for software

- 1046 architecture design with genetic algorithms. In: 2010 Fifth International Confer-
1047 ence on Software Engineering Advances, pp. 359–366 (2010). [https://doi.org/10.](https://doi.org/10.1109/ICSEA.2010.61)
1048 [1109/ICSEA.2010.61](https://doi.org/10.1109/ICSEA.2010.61)
- 1049 [31] Tian, Z., Yang, Y., Cheng, S.: Rm2dm: A tool for automatic generation of
1050 oo design models from requirements models. In: 2023 IEEE/ACM 45th Inter-
1051 national Conference on Software Engineering: Companion Proceedings (ICSE-
1052 Companion), pp. 36–40 (2023). [https://doi.org/10.1109/ICSE-Companion58688.](https://doi.org/10.1109/ICSE-Companion58688.2023.00020)
1053 [2023.00020](https://doi.org/10.1109/ICSE-Companion58688.2023.00020)
- 1054 [32] Vernon, V.: Implementing Domain-driven Design, pp. 1–20. Addison-Wesley, ???
1055 (2013)
- 1056 [33] Wooldridge, M., Ciancarini, P.: Agent-oriented software engineering: The state
1057 of the art. In: Ciancarini, P., Wooldridge, M.J. (eds.) Agent-Oriented Software
1058 Engineering, pp. 1–28. Springer, Berlin, Heidelberg (2001)